Assignment #4: Chat Client

Goal

In this assignment, you will deepen your understanding of network protocols by implementing a Chat Client program in C or C++. This client will connect to a server on the Internet which implements the protocol described in this document.

Note: You are building the Chat Client, the Chat Server is provided for you and is hosted in the cloud.

Technology

This project is designed for Visual Studio 2022 or Visual Studio 2026. Your submission will be tested on a DigiPen lab-machine equivalent: Windows 10 or 11 and latest update of Visual Studio 2022 / 2026.

Chat Client

Example code can be downloaded from

```
https://github.com/DigiPen-Network-Classes/cs260-chat-client
```

Or using git:

```
git clone https://github.com/DigiPen-Network-Classes/cs260-chat-client
```

Chat Server

The **chat server** software and documentation can be found here:

```
https://github.com/DigiPen-Network-Classes/cs260-chat-server-solution
```

The documentation for the chat server and protocol, along with how to run a server locally:

```
https://github.com/DigiPen-Network-Classes/cs260-chat-server-solution/blob/main/README.md
```

JSON for Modern C++

This project also uses JSON for Modern C++ (https://github.com/nlohmann/json) for creating and parsing JSON packets.

Example Usage

```
// Convert a string to JSON
nlohmann::json parsedJSON = nlohmann::json::parse(jsonString);

// Access a field on the JSON object
const std::string type = parsedJSON["type"];

// Set a field on the JSON object
```

```
parsedJSON["name"] = "Bob";

// Looping over an array in a JSON object
for (const nlohmann::json& element : packet["array"]) {
    // ...
}

// Convert JSON to a string
const std::string stringJSON = nlohmann::to_string(parsedJSON);
```

UI Framework

The provided project is built with a UI framework built using Clay and Raylib. To complete the assignment you will need to interact with the interface in some key places – see below.

Changing Screens

To change the screen shown in the window you can set `BaseScreen::CurrentScreen` to 0 (new connection screen) or 1 (chat room screen).

Displaying Error Messages

Both screens have an error alert popup you can display by pushing error strings with the following function:

```
this->m_errorAlert.Push("This is an error message");
```

Updating User List and Message List

The chat room screen has two vectors it uses to keep track of state, `m_messageHistory` and `m_connectedUsers`, the UI will automatically reflect any changes on the next render.

Getting User Input

There are two text inputs you will need to read from for this assignment, `m_nameInput` on the connection screen and `m_messageInput` on the chat room screen, you can access their content at `m_nameInput.m_content`.

Chat Protocol

All messages are JSON objects sent over TCP, each terminated by a newline character (\n). Every packet has a “type” field that identifies what kind of message it is.

Important: a single recv() call may return a partial message, multiple messages, or anything in between. Be sure to buffer received bytes and split on newlines to extract complete packets.

Connection and Authentication

Connecting to the server uses a three step handshake.

Step 1: Client sends LOGIN

```
{ "type": "LOGIN" }
```

Step 2: Server replies with AUTHENTICATE

The AUTHENTICATE message will include a one-time nonce, to be used to construct the token in the CONNECT response.

```
{ "type": "AUTHENTICATE", "nonce": "A1B2C3D4..." }
```

Step 3: Client replies with CONNECT

This should include the display name of the user and the token computed from the name, secret, and nonce.

```
{
  "type": "CONNECT",
  "name": "Alice",
  "token":"computed_token"
}
```

The token is computed using SHA-256 with:

```
token = sha256(name + secret_token + nonce)
```

Where `secret_token` is the value passed as the third command-line argument – our shared secret between client and server.

SHA-256

A SHA-256 helper (`sha256_hex`) is provided in sha256.h.

Step 4: Server replies with HELLO...

```
{ "type": "HELLO" }
```

...or an ERROR

The error includes a reason for why the authentication failed.

```
{"type": "ERROR","reason": "Name already taken"}
```

Server -> Client Packets

Type	Fields	Description
HELLO		Handshake complete, switch to the chat room screen
AUTHENTICATE	nonce	One-time nonce for token computation

ERROR	reason	Something went wrong
MESSAGE	user, content, timestamp	A chat message sent to the chat room (including your own)
SYSTEM	content, timestamp	A server generated notification (user joined/left)
CONNECTED_USERS	users (array of strings)	Full list of currently connected users, sent after you join and whenever the list changes
MESSAGE_HISTORY	messages (array of user, content, timestamp)	Recent message history, sent once right after HELLO
PING		Keep alive from the server, **you must reply with PONG or you will be disconnected**

Client -> Server Packets

Type	Fields	Description
LOGIN		Begin the connection handshake
CONNECT	name, token	Complete the handshake
CHAT	content	Send a message to the chat room
PONG		Reply to a PING to maintain connection
DISCONNECT		Gracefully leave before closing the socket. Note that terminating the connection without the appropriate DISCONNECT is an error!

Programming

The chat client will take 3 arguments from the command line – the IP address of the server, the port that it should connect to, and the token to use for authentication. The Chat protocol is implemented over TCP.

If any argument is not provided, or is invalid (IP address not parseable, port not in the range 1-65535, not an integer, etc.) then your program should print out a simple usage statement.

```
std::cerr << "Usage: " << argv[0] << " <ip-address> <port> <token>\n";
```

Example Usage

```
CS260_ChatClient.exe 143.198.49.114 10000 digipen-cs260
```

A window will open with a name input. Type a username and click “Connect” (or press enter). On a successful handshake, the application will transition to the chat room where message history and connected users are shown.

Tasks

The starter code is organized around a ConnectionManager that owns the socket and a background thread. The main (rendering) thread queues outgoing messages with `ConnectionManager::Send()` and retrieves incoming messages with `ConnectionManager::Poll()`. To complete this assignment, you will need to write these methods and wire them together with the UI.

Example Server Code

You can download the chat server executable to run locally and for testing:
https://github.com/DigiPen-Network-Classes/cs260-chat-server-solution/releases/latest

To run the server, you must create a .env file in the same directory as the server executable with the following content – the PORT is what the chat client should connect to, HTTP_PORT is the web interface to the server (no real functionality here, yet), and the AUTH_TOKEN is the shared secret that your client will need to compose a token.

.env

```
PORT=3000
HTTP_PORT=3001
AUTH_TOKEN=secrettoken
```

Testing

1. Build your project and run the executable using the server address, port, and token
2. Enter a username and click Connect
3. Send a message and verify that it appears in the chat history and your name appears in the sidebar
4. Verify you properly respond to pings (keep running for over a minute and stay connected)
5. Test error cases, such as an incorrect token, taken username, missing arguments, etc.

Testing vs. Shared Server

The chat server is running on cs260.meancat.com (Summer 2026 IP address: 143.198.49.114) on port 10000.

Example

```
$ CS260_ChatClient.exe 143.198.49.114 10000 digipen-cs260
```

Shared Secret

For Summer 2026, the secret on cs260.meancat.com is:

```
digipen-cs260
```

Testing with Demo Client

A complete client build is provided at
https://github.com/DigiPen-Network-Classes/cs260-chat-client/releases/tag/Demo-Build

If you are stuck wondering how to implement something.

Submission

Your project should be named CS260_ChatClient.exe. You should not do anything fancy with paths or copying binaries or using cmake, etc. It should be buildable "as is" using Visual Studio 2022/2026.

You should zip up the solution, project, and source files – but DO NOT include extra directories such as an extra top level directory, and DO NOT include build product (debug/release builds and folders).

You may include a README that contains any comments or questions – anything you think I should know about. This is optional.

Submission File Name

You must name the archive YourAlias-CS260-ChatClient.zip where "YourAlias" is the username used for Moodle.

Grading

From a base score of 100, you'll lose points for the following errors:

- -100 if your solution cannot be compiled
    - Do not rely on external tools like cmake!
- -100 if your client uses gets() instead of a safe input method. gets() is considered a compile-time error.
- -70 if your program does not send and receive messages correctly.
- Between -10 and -40 (depending on severity) if your program does not implement multithreading in a safe manner, if threads are used.
- -10 if your client does not parse command arguments correctly or allows invalid input.
- -10 if your code compiles with any warnings.
- (Various) Other penalties will be assessed depending on the accuracy of the output, handling error conditions correctly, and other programming mistakes.

Code Quality

Your program should look finished and presentable – no large blocks of commented code, informative variable and function names, and so on. Penalties will be assessed based on the severity of the presentation.

Your code must compile on all platforms and configurations included with the Solution, with no warnings at the default warning level of 3. Warnings will incur a 10-point penalty. Do not change the warning level for the assignment (unless otherwise specified) -- this includes Code Analysis warnings as well as build output, but not tasks or hints)

Submission Penalties

- (-10) if your submission is named incorrectly
- (-10) if your output is named incorrectly
- (-10) if your submission includes additional files or directories
- (-10) if your submission includes binaries